Lista de Exercícios — Dicionários em JavaScript

Oct 16, 2026 · @Tarik Ponciano

Orientações gerais

Um dicionário guarda dados em pares **chave: valor**. Em JavaScript, ele é escrito como um objeto entre chaves `{}`. Enquanto o array organiza os dados por posição (`lista[0]`), o dicionário organiza por nome (`aluno.nome`).

```
const aluno = { nome: 'Ana', idade: 17, curso: 'TDS' };

console.log(aluno.nome);       // lê uma propriedade: Ana
aluno.idade = 18;              // altera um valor
aluno.turma = 'A';             // adiciona uma nova chave
console.log(aluno['curso']);   // outra forma de ler: TDS
```

- Use o mesmo projeto Node.js da lista de arrays, com `@inquirer/prompts` e `"type": "module"` no `package.json`.
- Crie um arquivo por questão e execute com `node questao1.js`.
- Use `input()` para textos e `number()` para números, sempre com `await`.
- **Ponto (`obj.chave`):** quando você já sabe o nome da chave ao escrever o código.
- **Colchetes (`obj[variavel]`):** quando o nome da chave está guardado em uma variável, por exemplo, digitado pelo usuário.
- Exiba os resultados com `console.log()`.

Questão 1 — Ficha de cadastro

Faça um programa que peça o **nome**, a **idade** e a **cidade** de uma pessoa e guarde tudo em um único dicionário. Depois, exiba a ficha completa.

Requisitos

1. Colete os três dados com `input()` e `number()`.
2. Crie um objeto `pessoa` com as chaves `nome`, `idade` e `cidade`.
3. Exiba o objeto inteiro com `console.log(pessoa)`.
4. Exiba também cada dado em uma linha, acessando as propriedades com ponto.

Exemplo de execução

```
? Nome: Ana
? Idade: 17
? Cidade: Fortaleza

{ nome: 'Ana', idade: 17, cidade: 'Fortaleza' }

----- FICHA -----
Nome: Ana
Idade: 17 anos
Cidade: Fortaleza
```

Pense nisso: como você faria essa mesma ficha usando um array? Por que `pessoa.idade` é mais fácil de entender que `pessoa[1]`?

Questão 2 — Atualizando um produto

Uma loja precisa atualizar os dados de um produto. Comece com o dicionário abaixo já escrito no código:

```
const produto = { nome: 'Fone de ouvido', preco: 80, estoque: 12 };
```

Requisitos

1. Exiba os dados atuais do produto.
2. Pergunte o **novo preço** e altere a chave `preco`.
3. Pergunte **quantas unidades foram vendidas** e desconte do `estoque`.
4. Pergunte a **categoria** do produto e adicione uma nova chave `categoria`, que não existia no início.
5. Exiba os dados atualizados.
6. Se o estoque ficar zerado, exiba `Produto esgotado!`.

Exemplo de execução

```
Produto: Fone de ouvido | Preço: R$ 80 | Estoque: 12

? Novo preço: 75
? Unidades vendidas: 4
? Categoria: Eletrônicos

Produto: Fone de ouvido | Preço: R$ 75 | Estoque: 8 | Categoria: Eletrônicos
```

Pense nisso: o objeto foi declarado com `const`, mas mesmo assim foi possível alterar seus valores. Por quê?

Questão 3 — Controle de gastos do mês

Agora as chaves não estão escritas no código: o usuário é quem decide. Faça um programa que registre os gastos do mês, com o nome de cada categoria como chave e o valor gasto como valor.

Requisitos

1. Comece com um dicionário vazio: `const gastos = {};`.
2. Pergunte quantas categorias de gasto serão registradas.
3. Para cada categoria, peça o nome e o valor e guarde no dicionário usando colchetes: `gastos[categoria] = valor;`.
4. Percorra o dicionário com `for...in` e exiba cada categoria com seu valor.
5. Durante o mesmo laço, calcule o total gasto e descubra qual categoria teve o maior gasto.

Exemplo de execução

```
? Quantas categorias? 4
? Categoria: Alimentação
? Valor: 650
? Categoria: Transporte
? Valor: 180
? Categoria: Internet
? Valor: 99.90
? Categoria: Lazer
? Valor: 120

----- GASTOS DO MÊS -----
Alimentação: R$ 650.00
Transporte: R$ 180.00
Internet: R$ 99.90
Lazer: R$ 120.00
Total gasto: R$ 1049.90
Maior gasto: Alimentação (R$ 650.00)
```

Dica: no `for (const chave in gastos)`, a variável `chave` recebe o nome da categoria. Para pegar o valor, use `gastos[chave]`.

Pense nisso: o que acontece se o usuário digitar a mesma categoria duas vezes? Teste e depois altere o programa para somar o novo valor ao que já existe, em vez de substituir.

Questão 4 — Glossário de programação

Faça um glossário em que a chave é o **termo** e o valor é o **significado**. O programa funciona com menu e se repete até o usuário escolher sair.

```
===== GLOSSÁRIO =====
1 - Cadastrar termo
2 - Buscar termo
3 - Listar todos
0 - Sair
```

Requisitos

1. **Cadastrar:** peça o termo e o significado e guarde com `glossario[termo] = significado;`.
2. Se o termo já existir, exiba `Termo já cadastrado` e não substitua o significado.
3. **Buscar:** peça um termo e exiba o significado. Se não existir, exiba `Termo não encontrado`.
4. **Listar todos:** percorra com `for...in` e exiba cada termo com seu significado.
5. Ao listar, informe também quantos termos estão cadastrados.

Exemplo de execução (buscar)

```
? Termo: array
array: estrutura que guarda vários valores em sequência, acessados por
posição.

? Termo: loop
Termo não encontrado.
```

Dica: para verificar se uma chave existe, use o operador `in`: `if (termo in glossario)`.

Desafio extra: adicione a opção `4 - Remover termo` usando `delete glossario[termo];`.

Questão 5 — Carrinho de compras (lista + dicionário)

Um dicionário guarda os dados de **um** item. Uma lista guarda **vários** itens. Juntando os dois, temos uma lista de dicionários: cada posição do array é um objeto com os dados de um produto.

```
const carrinho = [
  { nome: 'Caderno', preco: 15, quantidade: 3 },
  { nome: 'Caneta', preco: 2.5, quantidade: 10 }
];

console.log(carrinho[0].nome);   // Caderno
console.log(carrinho.length);    // 2
```

Faça um programa de carrinho de compras com o menu abaixo, que se repete até o usuário escolher sair.

```
===== CARRINHO =====
1 - Adicionar produto
2 - Ver carrinho
0 - Finalizar compra
```

Requisitos

1. Comece com um array vazio: `const carrinho = [];`.
2. **Adicionar produto:** peça nome, preço e quantidade, monte um objeto com as chaves `nome`, `preco` e `quantidade` e adicione ao carrinho com `push()`.
3. **Ver carrinho:** percorra o array com `for` e exiba cada produto em lista numerada, com o subtotal (preço × quantidade).
4. Ao final da listagem, exiba a quantidade de itens no carrinho e o valor total da compra.
5. Se o carrinho estiver vazio, exiba `Carrinho vazio`.
6. **Finalizar compra:** exiba o total a pagar e encerre o programa.

Exemplo de execução (ver carrinho)

```
----- SEU CARRINHO -----
1. Caderno - 3 x R$ 15.00 = R$ 45.00
2. Caneta - 10 x R$ 2.50 = R$ 25.00
3. Mochila - 1 x R$ 120.00 = R$ 120.00
Itens no carrinho: 3
Total: R$ 190.00
```

Desafio extra 1: se o usuário adicionar um produto que já está no carrinho, some a quantidade no item existente em vez de criar outro.

Desafio extra 2: volte ao sistema de notas da lista de arrays e refaça com uma lista de dicionários. Troque os dois arrays paralelos `nomes` e `notas` por um único array de objetos `{ nome, nota, situacao }`.